



Server Features

Tools

📄 Copy page



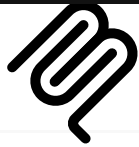
The Model Context Protocol (MCP) allows servers to expose tools that can be invoked by language models. Tools enable models to interact with external systems, such as querying databases, calling APIs, or performing computations. Each tool is uniquely identified by a name and includes metadata describing its schema.

❗ For brevity, the request examples on this page omit the `_meta` request metadata (`io.modelcontextprotocol/protocolVersion` , `io.modelcontextprotocol/clientInfo` , and `io.modelcontextprotocol/clientCapabilities`). Every request **MUST** include the required `_meta` fields; see [_meta](#) .

1. User Interaction Model

Tools in MCP are designed to be **model-controlled**, meaning that the language model can discover and invoke tools automatically based on its contextual understanding and the user's prompts.

However, implementations are free to expose tools through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.



Model Context Protocol

- Provide UI that makes clear which tools are being exposed to the AI model
- Insert clear visual indicators when tools are invoked
- Present confirmation prompts to the user for operations, to ensure a human is in the loop

2. Capabilities

Servers that support tools **MUST** declare the `tools` capability:

```
{
  "capabilities": {
    "tools": {
      "listChanged": true
    }
  }
}
```

`listChanged` indicates whether the server will emit notifications when the list of available tools changes.

Servers that declare the `tools` capability **MUST** respond to `tools/list` requests with the set of tools currently available to the requesting client. This set **MAY** be empty and **MAY** change over time (see [List Changed Notification](#)), but **MUST NOT** vary per-connection or as a side effect of other requests on the connection. The set **MAY** vary by the authorization presented on the request — for example, returning only the tools the caller's granted scopes permit — since credentials are per-request input, not connection state.

Servers **SHOULD** return tools in a deterministic order (i.e., the same ordering across requests when the underlying set of tools has not changed). Deterministic ordering enables clients to reliably cache the tool list and improves LLM prompt cache hit rates when tools are included in model context.



Model Context Protocol

To discover available tools, clients send a `tools/list` request. This operation supports pagination and caching.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

Response:



Model Context Protocol

```
"resultType": "complete",
"tools": [
  {
    "name": "get_weather",
    "title": "Weather Information Provider",
    "description": "Get current weather information for a location",
    "inputSchema": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "City name or zip code"
        }
      },
      "required": ["location"]
    },
    "icons": [
      {
        "src": "https://example.com/weather-icon.png",
        "mimeType": "image/png",
        "sizes": ["48x48"]
      }
    ]
  }
],
"nextCursor": "next-page-cursor",
"ttlMs": 300000,
"cacheScope": "public"
}
```

3.2 Calling Tools

To invoke a tool, clients send a `tools/call` request:

Request:



Model Context Protocol

```
"params": {
  "name": "get_weather",
  "arguments": {
    "location": "New York"
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "complete",
    "content": [
      {
        "type": "text",
        "text": "Current weather in New York:\nTemperature: 72°F\nConditions"
      }
    ],
    "isError": false
  }
}
```

3.3 Input Required Tool Results

Servers **MAY** respond to `tools/call` with an `InputRequiredResult` to indicate that additional input is needed before the tool call can be completed. This follows the multi round-trip requests mechanism.

When retrying the request with input responses, clients include `inputResponses` and, if provided by the server, `requestState` in the request parameters:



Model Context Protocol

```
"id": 2,
"result": {
  "resultType": "input_required",
  "inputRequests": {
    "github_login": {
      "method": "elicitation/create",
      "params": {
        "mode": "form",
        "message": "Please provide your GitHub username",
        "requestedSchema": {
          "type": "object",
          "properties": {
            "name": { "type": "string" }
          },
          "required": ["name"]
        }
      }
    }
  },
  "requestState": "eyJsb2NhdGlvbiI6Ik5ldyBZb3JrIn0..."
}
```

Retry with Input Responses:



Model Context Protocol

```
"params": {
  "name": "get_weather",
  "arguments": {
    "location": "New York"
  },
  "inputResponses": {
    "github_login": {
      "action": "accept",
      "content": {
        "name": "octocat"
      }
    }
  },
  "requestState": "eyJsb2NhdGlvbiI6Ik5ldyBZb3JrIn0..."
}
```

Note that the JSON-RPC `id` **MUST** be different between the initial request and the retry.

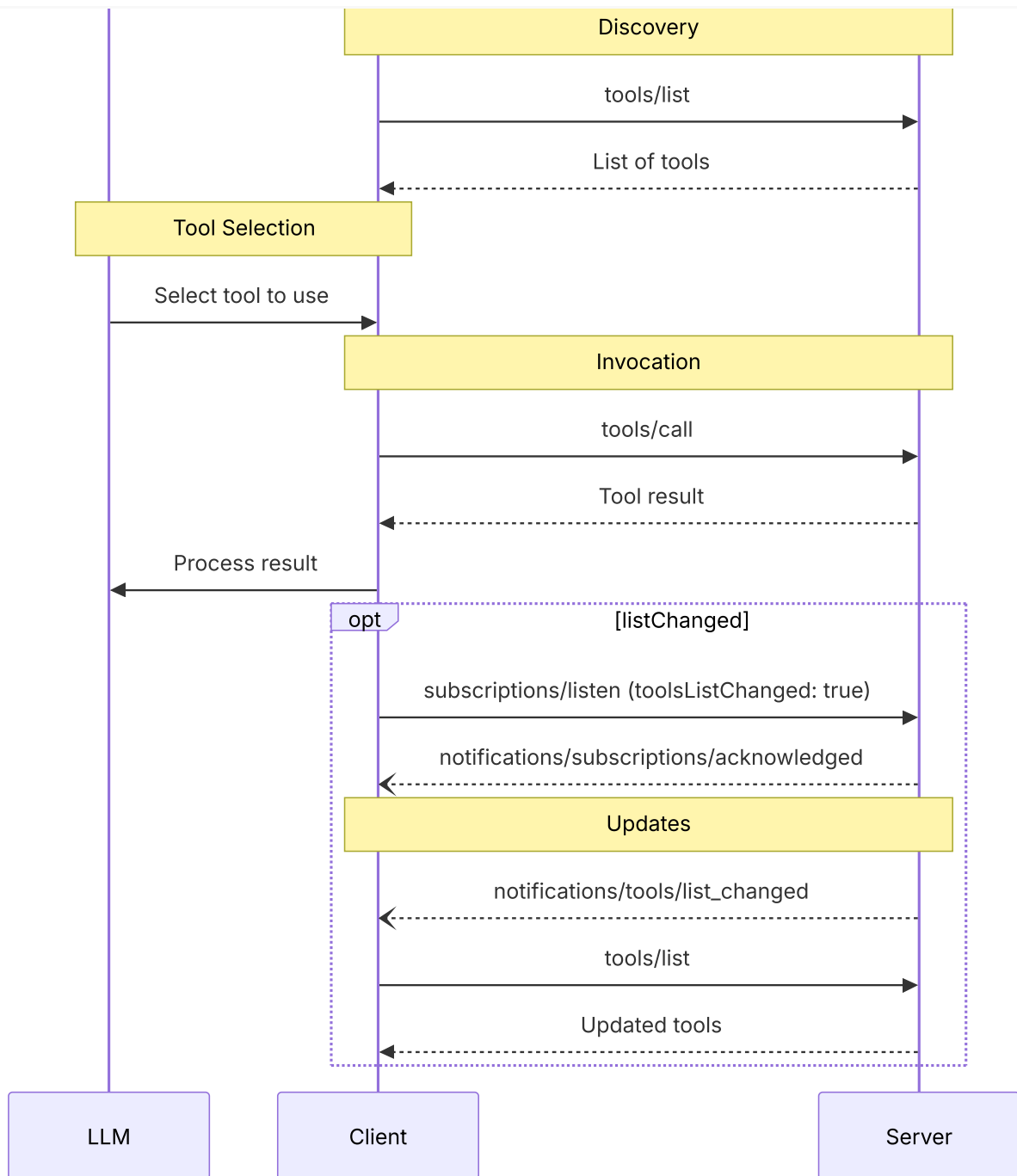
3.4 List Changed Notification

When the list of available tools changes, servers that declared the `listChanged` capability **SHOULD** send a notification to clients that have opened a `subscriptions/listen` stream with `toolsListChanged: true` :

```
{
  "jsonrpc": "2.0",
  "method": "notifications/tools/list_changed"
}
```



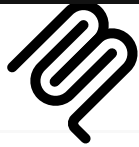
Model Context Protocol



5. Data Types

5.1 Tool

A tool definition includes:



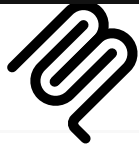
Model Context Protocol

- `icons` : Optional array of icons for display in user interfaces
- `inputSchema` : JSON Schema defining expected parameters
 - Follows the [JSON Schema usage guidelines](#)
 - Defaults to 2020-12 if no `$schema` field is present
 - **MUST** be a valid JSON Schema object (not `null`)
 - For tools with no parameters, use one of these valid approaches:
 - `{ "type": "object", "additionalProperties": false }` - **Recommended:** explicitly accepts only empty objects
 - `{ "type": "object" }` - accepts any object (including with properties)
 - Properties **MAY** include an [x-mcp-header](#) annotation to expose parameter values as HTTP headers
- `outputSchema` : Optional JSON Schema defining expected output structure
 - Follows the [JSON Schema usage guidelines](#)
 - Defaults to 2020-12 if no `$schema` field is present
- `annotations` : Optional properties describing tool behavior

⚠ For trust & safety and security, clients **MUST** consider tool annotations to be untrusted unless they come from trusted servers.

5.1.1 Tool Names

- Tool names **SHOULD** be between 1 and 128 characters in length (inclusive).
- Tool names **SHOULD** be considered case-sensitive.
- The following **SHOULD** be the only allowed characters: uppercase and lowercase ASCII letters (A-Z, a-z), digits (0-9), underscore (`_`), hyphen (`-`), and dot (`.`)
- Tool names **SHOULD NOT** contain spaces, commas, or other special characters.



Model Context Protocol

- `DATA_EXPORT_v2`
- `admin.tools.list`

❗ Tool name uniqueness is scoped to a single server. Clients or proxies that aggregate tools from multiple servers **MAY** encounter naming collisions (for example, two servers each exposing a `search` tool) and **SHOULD** implement a disambiguation strategy such as prefixing tool names with a server identifier.

The server `name` (from `serverInfo`) is not guaranteed to be unique across servers and **SHOULD NOT** be relied upon for disambiguation.

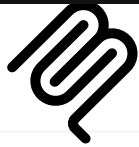
5.1.2 x-mcp-header

The `x-mcp-header` extension property allows servers to designate specific tool parameters to be mirrored into HTTP headers when using the **Streamable HTTP transport**. This enables network intermediaries (load balancers, proxies, WAFs) to route and process requests based on parameter values without parsing the request body.

The `x-mcp-header` property is placed directly within the JSON Schema of the property to be mirrored. Its value specifies the name portion of the resulting `Mcp-Param-{name}` HTTP header.

Constraints on `x-mcp-header` values:

- **MUST NOT** be empty
- **MUST** match HTTP field-name token syntax (`1*tchar` , **RFC 9110 Section 5.1**)
- **MUST NOT** contain control characters, including carriage return (CR, `\r`) or line feed (LF, `\n`)
- **MUST** be case-insensitively unique among all `x-mcp-header` values in the `inputSchema`



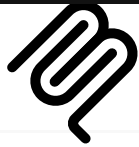
Model Context Protocol

- **MUST** only be applied to properties that are *statically reachable* from the schema root, as defined in [Custom Headers from Tool Parameters](#), which also defines how header values are extracted from call arguments

Clients using the Streamable HTTP transport **MUST** reject tool definitions where any `x-mcp-header` value violates these constraints. Rejection means the client **MUST** exclude the invalid tool from the result of `tools/list`. Clients **SHOULD** log a warning when rejecting a tool definition, including the tool name and the reason for rejection. This ensures that a single malformed tool definition does not prevent other valid tools from being used. Clients using other transports (e.g., stdio) **MAY** ignore `x-mcp-header` annotations entirely.

Example tool definition with `x-mcp-header` :

```
{
  "name": "execute_sql",
  "description": "Execute SQL on Google Cloud Spanner",
  "inputSchema": {
    "type": "object",
    "properties": {
      "region": {
        "type": "string",
        "description": "The region to execute the query in",
        "x-mcp-header": "Region"
      },
      "query": {
        "type": "string",
        "description": "The SQL query to execute"
      }
    },
    "required": ["region", "query"]
  }
}
```



Model Context Protocol

tokens, PII) with `x-mcp-header`, as header values are visible to network intermediaries.

5.2 Tool Result

Tool results may contain structured or **unstructured** content.

Unstructured content is returned in the `content` field of a result, and can contain multiple content items of different types:

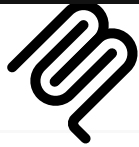
❗ All content types (text, image, audio, resource links, and embedded resources) support optional annotations that provide metadata about audience, priority, and modification times. This is the same annotation format used by resources and prompts.

5.2.1 Text Content

```
{
  "type": "text",
  "text": "Tool result text"
}
```

5.2.2 Image Content

```
{
  "type": "image",
  "data": "base64-encoded-data",
  "mimeType": "image/png",
  "annotations": {
    "audience": ["user"],
    "priority": 0.9
  }
}
```



Model Context Protocol

```
"data": "base64-encoded-audio-data",  
"mimeType": "audio/wav"  
}
```

5.2.4 Resource Links

A tool **MAY** return links to **Resources**, to provide additional context or data. In this case, the tool will return a URI that can be subscribed to or fetched by the client:

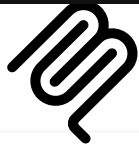
```
{  
  "type": "resource_link",  
  "uri": "file:///project/src/main.rs",  
  "name": "main.rs",  
  "description": "Primary application entry point",  
  "mimeType": "text/x-rust"  
}
```

Resource links support the same **Resource annotations** as regular resources to help clients understand how to use them.

- ① Resource links returned by tools are not guaranteed to appear in the results of a `resources/list` request.

5.2.5 Embedded Resources

Resources **MAY** be embedded to provide additional context or data using a suitable **URI scheme**. Servers that use embedded resources **SHOULD** implement the `resources` capability:



Model Context Protocol

```
"mimeType": "text/x-rust",
"text": "fn main() {\n    println!(\"Hello world!\");\n}",
"annotations": {
  "audience": ["user", "assistant"],
  "priority": 0.7,
  "lastModified": "2025-05-03T14:30:00Z"
}
}
```

Embedded resources support the same **Resource annotations** as regular resources to help clients understand how to use them.

5.2.6 Structured Content

Structured content is returned as a JSON value in the `structuredContent` field of a result. This can be any JSON value (object, array, string, number, boolean, or null) that conforms to the tool's `outputSchema` if one is defined.

For backwards compatibility, a tool that returns structured content SHOULD also return the serialized JSON in a TextContent block.

❗ `structuredContent` is server-produced result data and is unrelated to LLM “structured outputs” (schema-constrained model generation).

5.2.7 Output Schema

Tools may also provide an output schema for validation of structured results. If an output schema is provided:

- Servers **MUST** provide structured results that conform to this schema.
- Clients **SHOULD** validate structured results against this schema.



Model Context Protocol

```
"title": "Weather Data Retriever",
"description": "Get current weather data for a location",
"inputSchema": {
  "type": "object",
  "properties": {
    "location": {
      "type": "string",
      "description": "City name or zip code"
    }
  },
  "required": ["location"]
},
"outputSchema": {
  "type": "object",
  "properties": {
    "temperature": {
      "type": "number",
      "description": "Temperature in celsius"
    },
    "conditions": {
      "type": "string",
      "description": "Weather conditions description"
    },
    "humidity": {
      "type": "number",
      "description": "Humidity percentage"
    }
  },
  "required": ["temperature", "conditions", "humidity"]
}
```

Example valid response for this tool:



Model Context Protocol

```
"resultType": "complete",
"content": [
  {
    "type": "text",
    "text": "{\\"temperature\\": 22.5, \\"conditions\\": \\"Partly cloudy\\",
  }
],
"structuredContent": {
  "temperature": 22.5,
  "conditions": "Partly cloudy",
  "humidity": 65
}
}
```

Example tool with array output schema:



Model Context Protocol

```
"inputSchema": {
  "type": "object",
  "properties": {}
},
"outputSchema": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "name": { "type": "string" },
      "email": { "type": "string" }
    },
    "required": ["id", "name", "email"]
  }
}
```

Example valid response for a tool with array output:



Model Context Protocol

```
"resultType": "complete",
"content": [
  {
    "type": "text",
    "text": "Found 2 users: Alice (alice@example.com) and Bob (bob@example.com)"
  }
],
"structuredContent": [
  { "id": "1", "name": "Alice", "email": "alice@example.com" },
  { "id": "2", "name": "Bob", "email": "bob@example.com" }
]
}
```

Providing an output schema helps clients and LLMs understand and properly handle structured tool outputs by:

- Enabling strict schema validation of responses
- Providing type information for better integration with programming languages
- Guiding clients and LLMs to properly parse and utilize the returned data
- Supporting better documentation and developer experience



Model Context Protocol

```
{
  "name": "calculate_sum",
  "description": "Add two numbers",
  "inputSchema": {
    "type": "object",
    "properties": {
      "a": { "type": "number" },
      "b": { "type": "number" }
    },
    "required": ["a", "b"]
  }
}
```

5.3.2 Tool with explicit draft-07 schema:

```
{
  "name": "calculate_sum",
  "description": "Add two numbers",
  "inputSchema": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "type": "object",
    "properties": {
      "a": { "type": "number" },
      "b": { "type": "number" }
    },
    "required": ["a", "b"]
  }
}
```



Model Context Protocol

```
"description": "Returns the current server time",  
"inputSchema": {  
  "type": "object",  
  "additionalProperties": false  
}  
}
```

6. Stateful Tools

ⓘ This section is non-normative guidance for tool design. The protocol has no concept of a state handle; from the wire's perspective a handle is an ordinary string in a tool result and an ordinary argument to subsequent tool calls.

MCP has no protocol-level session, so a server cannot rely on implicit per-connection state to relate one tool call to the next. Servers that need to maintain state across calls — a shopping cart, an open browser context, a database transaction — should do so by returning an explicit handle from a creation tool and accepting that handle as an argument on subsequent calls.

For example, a server that manages a shopping cart might expose:



Model Context Protocol

```
{
  "content": [{ "type": "text", "text": "Created basket bsk_a1b2c3" }],
  "structuredContent": { "basket_id": "bsk_a1b2c3" }
}

// → tools/call
{
  "name": "add_item",
  "arguments": { "basket_id": "bsk_a1b2c3", "sku": "..." }
}
```

The model is responsible for carrying `basket_id` forward; the server stores the cart contents under that key and looks them up on each call.

When designing handles, servers should consider:

- **Authorization.** For authenticated servers, a handle is a name, not a capability. The server should validate the caller's authorization against the handle on every call. For unauthenticated servers, where the handle is necessarily a bearer token, it should be generated with sufficient entropy (e.g., a UUIDv4) and given a bounded lifetime.
- **Opacity.** Handles that encode internal structure invite parsing or guessing; opaque identifiers do not.
- **Lifetime.** Because handles outlive any single connection, the server's retention policy should be stated in the creation tool's description (e.g., "baskets expire after 24 hours of inactivity") so the model can see it when deciding to create state.
- **Expiry errors.** A call against an expired or unknown handle should return a tool execution error that says so, so the model can recover by creating a new one.

7. Error Handling

Tools use two error reporting mechanisms:



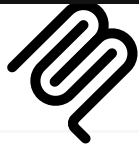
Model Context Protocol

- malformed requests (requests that fail to satisfy canToolRequestSchema),

- Server errors

They are returned as standard JSON-RPC errors:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "error": {
    "code": -32602,
    "message": "Unknown tool: invalid_tool_name"
  }
}
```



Model Context Protocol

- input validation errors (e.g., date in wrong format, value out of range,

- Business logic errors

They are reported in tool results with `isError: true` :

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "resultType": "complete",
    "content": [
      {
        "type": "text",
        "text": "Invalid departure date: must be in the future. Current c
      }
    ],
    "isError": true
  }
}
```

Clients **MAY** provide protocol errors to language models, though these are less likely to result in successful recovery. Clients **SHOULD** provide tool execution errors to language models to enable self-correction.

8. Security Considerations

1. Servers **MUST**:

- Validate all tool inputs
- Implement proper access controls
- Rate limit tool invocations
- Sanitize tool outputs



Model Context Protocol

accidental data exfiltration

- Validate tool results before passing to LLM
- Follow the `$ref` **resolution requirements** when validating tool inputs and outputs against `inputSchema` and `outputSchema`
- Implement timeouts for tool calls
- Log tool usage for audit purposes

Was this page helpful?

 Yes

 No



Copyright © Model Context Protocol a Series of LF Projects, LLC.
For web site terms of use, trademark policy and other project policies please see
<https://lfprojects.org>.